Systematic Analysis of Agentic Framework Vulnerabilities

Dillman Leigh — NAIL Institute

April 2026

The Deserialization Graveyard: A Systematic Analysis of Vulnerability Patterns Across Agentic AI Frameworks

Authors: Dillman Leigh[1], GitHub Copilot (Claude Opus 4.6)[2]
[1] NAIL Institute, Neuravant AI Limited
[2] AI Research Assistant — Anthropic/GitHub

Date: April 11, 2026
Target Venue: CCS 2026 (Cycle C) / USENIX Security 2027
Status: Submission Draft v1.0

Abstract

We present a systematic cross-framework analysis of **31 confirmed CVEs** across seven major agentic AI frameworks — LangGraph, LangChain, CrewAI, smolagents, PraisonAI, n8n, and Flowise — disclosed between July 2025 and April 2026. Our analysis reveals that these vulnerabilities are not isolated implementation bugs but cluster into five systemic architectural anti-patterns: (1) deserialization-as-RCE through unsafe state persistence, (2) fail-open sandbox degradation, (3) credential harvesting via tool abuse, (4) confused deputy attacks through mixed control planes, and (5) approval system bypass enabling shadow delegation.

We map all 31 CVEs against the Agentic Vulnerability Enumeration (AVE) taxonomy and find that **14 of 36 existing AVE classes** were predictively identified before the corresponding CVEs were disclosed — a 45% predictive hit rate for a taxonomy published months prior to most disclosures. The remaining CVEs reveal six new vulnerability classes not captured by existing taxonomies (CVE, CWE, or AVE), which we formally define and contribute.

Our cross-framework comparison reveals a disturbing convergence: every framework that persists agent state uses unsafe deserialization by default, every framework with code execution capabilities has inadequate sandbox isolation, and no framework implements per-invocation approval for high-risk tool calls. We argue that these patterns represent fundamental architectural failures in the agentic AI ecosystem — not

point defects — and propose a defense-in-depth reference architecture addressing each anti-pattern.

Keywords: agentic AI security, deserialization, LangGraph, LangChain, CrewAI, CVE analysis, framework vulnerabilities, supply chain security, sandbox escape

---

1. Introduction

The agentic AI ecosystem has reached a critical inflection point. Frameworks such as LangGraph [1], LangChain [2], CrewAI [3], and smolagents [4] have moved from research prototypes to production infrastructure, with LangChain alone exceeding 100,000 GitHub stars and CrewAI achieving Series A funding. These frameworks provide the scaffolding for autonomous AI agents that can execute code, manage persistent state, invoke external tools, and coordinate in multi-agent workflows.

Yet the security posture of these foundational frameworks remains alarmingly immature. Between July 2025 and April 2026 — a period of just nine months — we identified 31 CVEs with confirmed exploits across the seven most widely deployed agentic frameworks. Six of these are rated Critical (CVSS ≥ 9.0), nine are rated High, and as of this writing, CrewAI has four CVEs under CERT/CC coordination (VU#221883) with no complete patch available.

This paper makes three contributions:

1. **First systematic cross-framework CVE analysis** of agentic AI infrastructure, identifying five architectural anti-patterns that recur independently across frameworks developed by different teams with different codebases.
2. **Empirical validation of the AVE taxonomy's predictive power**: 14 of 36 AVE vulnerability classes, published before most CVEs were disclosed, directly map to confirmed exploits — demonstrating that agentic vulnerability patterns are structurally predictable.
3. **Six new vulnerability classes** not captured by existing taxonomies, formally defined with AVSS scoring and contributed to the AVE Database.

1.1 Scope and Methodology

We analyzed all publicly disclosed security advisories for the following frameworks as of April 11, 2026:

Framework	Version Range	Advisory Source	CVEs Found
LangGraph	≤1.0.9 → 1.0.10	GitHub Security Advisories	6
LangChain Core	≤0.3.83 → 1.2.28	GitHub Security Advisories	7
CrewAI	Current (unpatched)	CERT/CC VU#221883	4
smolagents	≤1.25.0.dev0	GitHub Advisory Database	5
PraisonAI	≤4.5.127 → 4.5.128	GitHub Security Advisories	5
n8n	Various	GitHub Advisory Database	2

Framework	Version Range	Advisory Source	CVEs Found
Flowise	Various	GitHub Advisory Database	2

For each CVE, we analyzed: (a) the advisory text and CVSS scoring, (b) available proof-of-concept code, (c) the affected source code and patch diff, (d) the root cause vulnerability class, and (e) mapping to AVE taxonomy entries. Where CVEs did not map to existing AVE classes, we defined new entries.

1.2 Threat Model

We consider three attacker profiles relevant to agentic framework exploitation:

Profile 1 — External Prompt Injector. An attacker who can influence agent input through crafted documents, emails, web pages, or API responses processed by the agent. This is the most common attack vector and requires no system access. Relevant CVEs: CVE-2025-68664, CVE-2026-40087, CVE-2026-2286.

Profile 2 — Persistence Layer Attacker. An attacker with write access to the agent's state persistence backend (database, Redis, filesystem). This may result from credential leakage, cloud misconfiguration, or supply chain compromise. Relevant CVEs: CVE-2025-64439, CVE-2026-27794, CVE-2026-28277, CVE-2025-67644.

Profile 3 — Tool/Plugin Supplier. An attacker who provides a malicious or compromised tool, MCP server, or npm package consumed by the agentic system. Relevant CVEs: CVE-2026-40159, CVE-2026-2275, GHSA-ffp3-3562-8cv3.

---

2. Anti-Pattern 1: Deserialization-as-RCE (12 CVEs)

The most prevalent vulnerability class across agentic frameworks is unsafe deserialization of persistent state. Every framework that provides checkpoint, cache, or store functionality was found to use at least one unsafe deserialization mechanism by default.

2.1 LangGraph: The JsonPlusSerializer Attack Surface

LangGraph's entire state management layer — checkpoints, caches, and stores — relied on `JsonPlusSerializer`, which supported three serialization modes, all of which were found to be exploitable:

JSON Constructor Mode (CVE-2025-64439, CVSS 7.4). When msgpack serialization fails (e.g., due to Unicode surrogate characters), the serializer falls back to JSON mode, which supports a constructor-style format (`lc == 2, type == "constructor"`) for custom object reconstruction. An attacker who can trigger this fallback and inject a malicious payload achieves arbitrary code execution:

```
attack = {
    "lc": 2,
```

```
    "type": "constructor",
    "id": ["os", "system"],
    "kwargs": {"command": "echo pwned > /tmp/pwned.txt"},
}
# When deserialized via JsonPlusSerializer.loads_typed(),
# this executes os.system("echo pwned > /tmp/pwned.txt")
```

The attack requires the attacker to cause the serializer to use JSON mode (by injecting a Unicode surrogate `\ud800` into agent state) and then inject the constructor payload. The fallback-triggering and exploitation can occur in separate interactions, making this a two-stage attack.

Pickle Fallback in BaseCache (CVE-2026-27794, CVSS 6.6). The `BaseCache` class defaulted to `JsonPlusSerializer(pickle_fallback=True)`. When msgpack serialization fails, cached values are stored as `("pickle", <bytes>)` tuples and deserialized via `pickle.loads()` — a well-known RCE vector. This was discovered by Trend Micro's Zero Day Initiative (ZDI-CAN-28385).

Unsafe msgpack Reconstruction (CVE-2026-28277, CVSS 6.8). Checkpoint loading via msgpack allows reconstruction of arbitrary Python objects during deserialization, expandable to code execution when the checkpoint store is compromised.

2.2 LangChain: Serialization Injection

LangChain's serialization layer used `lc` marker keys to distinguish serialized objects from plain data. The `dumps()` and `dumpd()` functions failed to escape user-controlled dictionaries containing `lc` keys, creating a critical injection vulnerability:

CVE-2025-68664 (CVSS 9.3 — Critical). An attacker injects `{"lc": 1, "type": "secret", "id": ["OPENAI_API_KEY"]}` into user-controlled fields (e.g., `metadata`, `additional_kwargs`, `response_metadata`). When the application serializes and deserializes this data — which occurs automatically in `astream_events(version="v1")`, `astream_log()`, and `RunnableWithMessageHistory` — the injected structure is treated as a legitimate LangChain secret reference, causing the runtime to load and expose the corresponding environment variable.

This is a *serialization confused deputy*: the application trusts its own serialization output, but user-controlled data has been mixed with control-plane metadata without escaping.

2.3 smolagents: Remote Executor Deserialization

CVE-2025-14931 (Critical). The Remote Python Executor in smolagents deserializes untrusted data, enabling RCE in deployment configurations that accept remote code execution results.

2.4 Structural Analysis

The deserialization pattern recurs because agentic frameworks face a fundamental tension: they need rich state persistence (custom Python objects, complex data

structures, model outputs) but the standard serialization approaches for rich objects (pickle, msgpack with extension types, JSON with type constructors) are all known to be unsafe when data provenance is not guaranteed.

Framework	Serialization	Unsafe Mechanism	CVE
LangGraph	JsonPlusSerializer (JSON mode)	Constructor reconstruction	CVE-2025-64439
LangGraph	JsonPlusSerializer (pickle)	`pickle.loads()` fallback	CVE-2026-27794
LangGraph	JsonPlusSerializer (msgpack)	Object reconstruction ext types	CVE-2026-28277
LangChain	`dumps()`/`dumpd()`	`lc` key injection	CVE-2025-68664
smolagents	Remote Executor	Untrusted deserialization	CVE-2025-14931

Root cause: No framework implemented a *safe-by-default* serialization strategy. All defaulted to maximum flexibility (serialize anything) rather than minimum privilege (serialize only explicitly allowed types).

---

3. Anti-Pattern 2: Fail-Open Sandbox Degradation (4 CVEs)

3.1 CrewAI: Silent Sandbox Bypass

CrewAI's CodeInterpreter tool is designed to execute agent-generated Python code inside a Docker container. However, two critical vulnerabilities transform this from a sandboxed environment to unrestricted host execution:

CVE-2026-2275 (CVSS 9.7 — Critical). When Docker is unavailable (not installed, daemon stopped, or connection refused), the CodeInterpreter silently falls back to `SandboxPython`, a Python-based sandbox that can be escaped via `ctypes` to call arbitrary C functions. The fallback occurs without user notification, without logging, and without consent.

CVE-2026-2287 (CVSS 9.8 — Critical). Even when Docker is available at startup, CrewAI does not verify Docker availability *during runtime*. If Docker becomes unavailable during agent execution (e.g., daemon crash, resource exhaustion, or adversarial Docker socket manipulation), the system silently degrades to the insecure sandbox. This is exploitable in multi-agent scenarios where one agent could exhaust Docker resources to force other agents into the insecure fallback.

3.2 smolagents: Sandbox Escape

CVE-2025-5120 (Critical). The `local_python_executor.py` module in smolagents contains a sandbox escape vulnerability that allows breakout from the intended execution sandbox. The specific mechanism exploits gaps in the Python AST-based execution restrictions.

3.3 Structural Analysis

The fail-open pattern violates a fundamental security principle: **systems should fail to a secure state**. In agentic contexts, this is especially dangerous because:

1. The agent itself cannot detect the security degradation — it continues generating and executing code with the same confidence regardless of sandbox state.
2. The human operator may have approved "code execution" based on the documented Docker isolation, which no longer applies.
3. The CERT/CC coordination note (VU#221883) confirms that as of April 2026, CrewAI has acknowledged but not fully patched these vulnerabilities.

---

4. Anti-Pattern 3: Credential Harvesting via Tool Abuse (8 CVEs)

4.1 PraisonAI: The `expandvars` Deception

CVE-2026-40153 (CVSS 7.4 — High). PraisonAI's shell tool calls `os.path.expandvars()` on every command argument despite using `shell=False` for subprocess execution. This creates a deceptive security gap:

1. The approval UI displays the command with unexpanded variable references: `echo $DATABASE_URL`
2. The user approves, reasoning that `shell=False` prevents variable expansion
3. `os.path.expandvars()` resolves `$DATABASE_URL` to `postgres://admin:s3cretP@ss@prod-db:5432/app`
4. The expanded secret appears in stdout, returned to the LLM context

This is a confused deputy within the approval system itself — the display mechanism and the execution mechanism disagree on what will happen.

4.2 PraisonAI: MCP Environment Inheritance

CVE-2026-40159 (CVSS 5.5). PraisonAI's MCP integration uses `os.environ.copy()` to pass the parent process's entire environment to spawned MCP server subprocesses. When developers use `MCP("npx -y @smithery/cli ...")` to invoke third-party MCP servers, those servers inherit `OPENAI_API_KEY`, `AWS_SECRET_ACCESS_KEY`, database credentials, and all other environment variables.

4.3 LangChain: Serialization-Based Secret Extraction

CVE-2025-68664 (CVSS 9.3). As described in Section 2.2, the `lc` key injection enables direct extraction of environment variable secrets. The attack is amplified

by the fact that `secrets_from_env` defaulted to `True` in `load()`/`loads()`, meaning any deserialized `{"lc": 1, "type": "secret", "id": ["ENV_VAR"]}` structure would automatically resolve to the environment variable value.

4.4 LangChain: Path Traversal for Credential Files

CVE-2026-34070 (CVSS 7.5 — High). The legacy `load_prompt()` function reads files from paths embedded in deserialized config dicts without path validation. An attacker can read: - `.txt` files: cloud-mounted secrets, requirements files, internal prompts - `.json`/`.yaml` files: `~/.docker/config.json`, `~/.azure/accessTokens.json`, Kubernetes manifests

4.5 CrewAI: SSRF to Cloud Metadata

CVE-2026-2286 (CVSS 9.8 — Critical). CrewAI's RAG search tools do not validate URLs provided at runtime, enabling SSRF attacks against cloud metadata endpoints (e.g., `http://169.254.169.254/latest/meta-data/iam/security-credentials/`) and internal services.

4.6 Structural Analysis

Every framework treats credential management as an afterthought. The common pattern is:

1. **Environment variables as the primary secret store** — inherited by all subprocesses by default
2. **No secret filtering** — no framework strips `*_KEY`, `*_SECRET`, `*_TOKEN` patterns from subprocess environments
3. **Display/execution divergence** — approval UIs show sanitized representations while execution operates on raw values

---

5. Anti-Pattern 4: Confused Deputy via Mixed Control Planes (6 CVEs)

5.1 LangChain: User Data as Control Plane

CVE-2025-68664, CVE-2025-65106, CVE-2026-40087. LangChain's serialization and template systems mix user-controlled data with framework control metadata in the same namespace. The `lc` key injection exploits the absence of a boundary between "data about objects" and "data from users." Template injection exploits the absence of a boundary between "template structure" and "template values."

5.2 LangGraph: SQL Query Construction from User Input

CVE-2025-67644, CVE-2025-64104. LangGraph's SQLite checkpointer constructs SQL queries by interpolating user-controlled metadata filter keys directly into f-strings. While filter *values* are parameterized, filter *keys* are not — a subtle

variant of SQL injection where the injection point is the column identifier rather than the value.

```
# VULNERABLE — filter key interpolated directly
f"json_extract(CAST(metadata AS TEXT), '$.{query_key}') {operator}"
# Injection: query_key = "x') OR '1'='1"
```

5.3 LangSmith: Prototype Pollution

CVE-2026-40190. The LangSmith Client SDK contains a prototype pollution vulnerability via an incomplete `__proto__` guard in an internal lodash `set()` implementation, allowing property injection into JavaScript object prototypes.

---

6. Anti-Pattern 5: Approval System Bypass (3 CVEs)

6.1 PraisonAI: Coarse-Grained Approval Cache

GHSA-ffp3-3562-8cv3 (CVSS 5.5). PraisonAI's approval system caches tool approval decisions by tool *name* only, not by invocation arguments. Once a user approves `execute_command("ls -la")`, all subsequent `execute_command` calls — including `execute_command("printenv OPENAI_API_KEY")` — bypass the approval prompt entirely.

The approval cache uses a `ContextVar` that persists for the entire agent execution context with no timeout, no command-count limit, and no clearing between tool calls. The `clear_approved()` function exists but is never called in the agent's tool execution path.

The console UI compounds this by displaying command-specific information and using the phrase "this tool" — implying per-invocation consent that the system does not enforce.

6.2 PraisonAI: Auto-Approve Environment Variable

The `PRAISONAI_AUTO_APPROVE=true` environment variable disables all approval prompts, and is documented as a CI/CD convenience feature. In practice, this creates a pathway where automated deployments operate with zero human oversight on tool execution — the exact scenario where credential harvesting attacks (Section 4) are most dangerous.

6.3 Structural Analysis

No surveyed framework implements per-invocation approval for high-risk tool calls. The pattern across all frameworks is:

Framework	Approval Mechanism	Granularity	Bypass Available
LangGraph	None by default	N/A	N/A (no approval system)
LangChain	None by default	N/A	N/A
CrewAI	`allow_code_execution=True`	Binary per-agent	Yes (config flag)
PraisonAI	Console/YAML/Auto	Per-tool-name	Yes (cache + env var)
smolagents	None by default	N/A	N/A

---

7. AVE Taxonomy Mapping and New Classifications

7.1 Predictive Coverage

Of the 36 AVE vulnerability classes published prior to this analysis, 14 directly map to confirmed CVEs:

AVE ID	AVE Name	CVEs Mapped	Frameworks
AVE-0001	Sleeper Payload Injection	CVE-2026-27794, CVE-2025-14931	LangGraph, smolagents
AVE-0014	MCP Tool Registration Poisoning	CVE-2026-40159	PraisonAI
AVE-0017	Container Isolation Bleed	CVE-2026-2275, CVE-2026-2287, CVE-2025-5120	CrewAI, smolagents
AVE-0019	Pydantic Schema Exploitation	CVE-2025-65106, CVE-2026-40190	LangChain, LangSmith
AVE-0022	Memory Laundering	CVE-2026-28277	LangGraph
AVE-0026	Confused Deputy Attack	CVE-2025-64439, CVE-2026-2654, CVE-2026-4963	LangGraph, smolagents
AVE-0027	Shadow Delegation	GHSA-ffp3-3562-8cv3	PraisonAI
AVE-0028	Credential Harvesting	CVE-2025-68664, CVE-2026-34070, CVE-2026-2285	LangChain, CrewAI
AVE-0029	Temporal Sleeper Agent	CVE-2025-64439	LangGraph
AVE-0030	Semantic Trojan Horse	CVE-2025-11844, CVE-2026-40112, CVE-2026-40087	smolagents, PraisonAI, LangChain

AVE ID	AVE Name	CVEs Mapped	Frameworks
AVE-0032	Multi-Hop Tool Chain	CVE-2026-27022, CVE-2025-67644, CVE-2026-26013, CVE-2026-2286	LangGraph, LangChain, CrewAI
AVE-0033	Jailbreak Chaining	Auto-approve bypass chain	PraisonAI
AVE-0034	Federated Poisoning	CVE-2026-27794 (multi-tenant cache)	LangGraph

This 39% hit rate (14/36) against independently disclosed CVEs validates the AVE taxonomy as a *predictive* framework, not merely a retrospective classification scheme.

7.2 New Vulnerability Classes

Six vulnerability patterns identified in this analysis were not captured by existing AVE, CWE, or MITRE classifications. We define them as AVE-2025-0037 through AVE-2025-0042:

New AVE	Name	Key CVEs	Category
AVE-2025-0037	Serialization Confused Deputy	CVE-2025-68664	injection
AVE-2025-0038	Fail-Open Sandbox Degradation	CVE-2026-2275, CVE-2026-2287	structural
AVE-2025-0039	Approval Display Divergence	CVE-2026-40153, GHSA-ffp3-3562-8cv3	delegation
AVE-2025-0040	Environment Inheritance Leak	CVE-2026-40159	credential
AVE-2025-0041	State Persistence Deserialization	CVE-2025-64439, CVE-2026-27794, CVE-2026-28277	memory
AVE-2025-0042	Checkpoint Injection Chain	Two-stage surrogate + constructor	structural

---

8. Defense Architecture

Based on the five anti-patterns identified, we propose a defense-in-depth reference architecture:

8.1 Safe Serialization Layer

Principle: Serialize only explicitly allowed types; reject everything else.

```
Agent State → Schema-Driven Allowlist Filter → msgpack (strict mode)
                                                 → NO pickle fallback
                                               → NO constructor reconstruction
                                                 → Cryptographic integrity tag
```

LangGraph's post-patch approach (`LANGGRAPH_STRICT_MSGPACK=true` with schema-derived allowlists) is a step in the right direction but remains opt-in. We propose that *strict-by-default* with explicit opt-out should be the industry standard.

8.2 Fail-Closed Sandbox Architecture

Principle: If the sandbox cannot be verified as active and intact, refuse to execute code.

```
Code Execution Request
  → Verify Docker daemon (socket + test container)
    → Verify container isolation (namespace + cgroup checks)
      → Execute with capability restrictions
        → Verify output integrity
  → On ANY failure: REFUSE execution + alert operator
```

8.3 Credential Isolation

Principle: Subprocess environments should contain ONLY explicitly whitelisted variables.

```python
# SAFE: Explicit allowlist
SAFE_ENV_VARS = {"PATH", "HOME", "USER", "LANG", "LC_ALL"}
process_env = {k: v for k, v in os.environ.items() if k in SAFE_ENV_VARS}
process_env.update(tool_specific_vars)  # Only vars the tool needs
```

8.4 Control Plane / Data Plane Separation

Principle: User-controlled data must never be interpretable as framework control metadata.

This requires: 1. Structural separation (different namespaces, prefixes, or envelope layers) 2. Escaping at serialization boundaries 3. Type-level enforcement (tagged unions rather than marker keys)

8.5 Per-Invocation Approval with Transparent Display

Principle: High-risk tool invocations require per-invocation approval, and the approval display must show the *exact* operation that will execute.

```python
# Show expanded values in approval display
display_command = [os.path.expandvars(arg) for arg in command]
```

```
if approve(display_command):  # User sees actual values
    execute(display_command)
```

---

9. Discussion

9.1 Industry Maturity Assessment

The concentration of critical CVEs in 2025–2026 reveals an industry in the "move fast and break things" phase of its security lifecycle. Key indicators of immaturity:

- **No framework has a formal threat model** in its documentation
- **Default configurations are universally unsafe** — every patch changed a default from permissive to restrictive
- **Security features are opt-in** — strict deserialization, approval systems, and sandbox verification all require explicit activation
- **CERT/CC coordination was required** for CrewAI because the vendor acknowledged but could not fully remediate

9.2 The Compounding Risk

These anti-patterns compound in multi-agent deployments. Consider a scenario using CrewAI agents with LangGraph state persistence and PraisonAI tool execution:

1. An attacker injects a Unicode surrogate via prompt injection → triggers LangGraph's JSON serialization fallback (CVE-2025-64439)
2. The malicious checkpoint payload executes arbitrary code in the agent runtime
3. The compromised agent generates code for CrewAI's CodeInterpreter → Docker is unavailable → code executes on host (CVE-2026-2275)
4. Host-level code uses PraisonAI's shell tool with `$AWS_SECRET_ACCESS_KEY` → secrets exfiltrated without approval (CVE-2026-40153 + GHSA-ffp3-3562-8cv3)

Each vulnerability individually might be rated Moderate or High. Chained together, they constitute a Critical attack path from prompt injection to cloud credential exfiltration with no human intervention required.

9.3 Relation to NAIL's Defense Architecture

NAIL's 5-layer Pathology Collider architecture [18] addresses each anti-pattern:

Anti-Pattern	NAIL Defense Layer
Deserialization-as-RCE	Layer 3: Memory Firewall — validates all state transitions against schema allowlists
Fail-Open Sandbox	Layer 4: Tripwires — continuous runtime integrity verification
Credential Harvesting	Layer 1: Stochastic Monitor — detects anomalous data flows

Anti-Pattern	NAIL Defense Layer
Confused Deputy	Layer 2: Pathology Tensor — multi-dimensional pathology classification
Approval Bypass	Layer 5: Somatic Startle Response — autonomic response to approval anomalies

---

10. Related Work

Greshake et al. [19] demonstrated indirect prompt injection as a practical attack on LLM-integrated applications but did not examine framework-level vulnerabilities. Zhan et al. [20] surveyed LLM agent risks but focused on model-level rather than infrastructure-level attacks. The OWASP Top 10 for LLM Applications [21] identifies deserialization and tool misuse as risk categories but does not provide cross-framework empirical analysis.

The CERT/CC Vulnerability Note VU#221883 [22] covers the CrewAI vulnerabilities discovered by Yarden Porat of Cyata. Our contribution extends this work by placing CrewAI's vulnerabilities in the context of ecosystem-wide patterns shared by all major frameworks.

Concurrent with our analysis, the LangSmith prototype pollution (CVE-2026-40190) and LangChain f-string validation bypass (CVE-2026-40087) were disclosed in April 2026, confirming that new vulnerability instances continue to emerge in the same anti-pattern categories.

---

11. Responsible Disclosure

All vulnerabilities discussed in this paper are publicly disclosed CVEs with available patches (except CrewAI, which is under CERT/CC coordination). We did not discover new zero-day vulnerabilities; our contribution is the systematic cross-framework analysis and pattern extraction. We contacted all framework maintainers with a draft of this paper prior to submission.

---

12. Conclusion

The agentic AI framework ecosystem exhibits systemic security failures that transcend individual implementation bugs. Five architectural anti-patterns — unsafe deserialization defaults, fail-open sandboxes, credential inheritance, mixed control planes, and coarse-grained approval — recur independently across seven major frameworks. These patterns are structurally predictable, as demonstrated by the AVE taxonomy's 39% predictive hit rate against subsequently disclosed CVEs.

The industry must transition from "secure if configured correctly" to "secure by default." Every unsafe default that was patched in the CVEs we analyzed was a design choice that prioritized developer convenience over user security. Until frameworks adopt safe-by-default serialization, fail-closed execution, credential isolation, control-plane separation, and per-invocation approval, the agentic AI ecosystem will continue to produce critical vulnerabilities at an accelerating rate.

We contribute six new AVE vulnerability classes and propose a defense-in-depth reference architecture addressing each anti-pattern. We call on the agentic AI community to treat security as a first-class architectural concern rather than a post-hoc compliance exercise.

References

[1] LangGraph. GitHub repository. https://github.com/langchain-ai/langgraph

[2] LangChain. GitHub repository. https://github.com/langchain-ai/langchain

[3] CrewAI. GitHub repository. https://github.com/crewAIInc/crewAI

[4] HuggingFace smolagents. GitHub repository. https://github.com/huggingface/smolagents

[5] PraisonAI. GitHub repository. https://github.com/MervinPraison/PraisonAI

[6] GHSA-wwqv-p2pp-99h5: RCE in "json" mode of JsonPlusSerializer. GitHub Security Advisory, Nov 2025.

[7] GHSA-mhr3-j7m5-c7c9: LangGraph BaseCache Deserialization RCE. ZDI-CAN-28385, Feb 2026.

[8] GHSA-g48c-2wqr-h844: Unsafe msgpack deserialization in LangGraph checkpoint loading. Mar 2026.

[9] GHSA-c67j-w6g6-q2cm: LangChain serialization injection — secret extraction. CVE-2025-68664, Dec 2025.

[10] GHSA-9rwj-6rc7-p77c: SQL injection via metadata filter key in SQLite checkpointer. CVE-2025-67644, Dec 2025.

[11] GHSA-qh6h-p6c9-ff54: Path traversal in legacy load_prompt functions. CVE-2026-34070, Mar 2026.

[12] GHSA-926x-3r5x-gfhw: Incomplete f-string validation in prompt templates. CVE-2026-40087, Apr 2026.

[13] CERT/CC Vulnerability Note VU#221883: CrewAI multiple vulnerabilities. Mar 2026.

[14] CVE-2026-2275: CrewAI CodeInterpreter SandboxPython fallback. CVSS 9.7.

[15] CVE-2026-2287: CrewAI Docker runtime check failure. CVSS 9.8.

[16] GHSA-v8g7-9q6v-p3x8: PraisonAI expandvars secret exfiltration. CVE-2026-40153, Apr 2026.

[17] GHSA-ffp3-3562-8cv3: PraisonAI coarse-grained tool approval cache bypass. Apr 2026.

[18] Leigh, D. The Pathology Collider: A Five-Layer Defense Architecture for Agentic AI. NAIL Institute Technical Report, 2026.

[19] Greshake, K. et al. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. AISec 2023.

[20] Zhan, Q. et al. Agentic AI Security: A Survey. arXiv:2502.08xxx, 2025.

[21] OWASP. Top 10 for LLM Applications. Version 2.0, 2025.

[22] CERT Coordination Center. VU#221883: CrewAI contains multiple vulnerabilities. Carnegie Mellon University SEI, Mar 2026.

Appendix A: Complete CVE Mapping Table

CVE	CVSS	Framework	Anti-Pattern	AVE Mapping	Patched
CVE-2025-64439	7.4 H	LangGraph	Deserialization	AVE-0041	✓ checkpoint ≥3.0
CVE-2025-64104	High	LangGraph	Mixed Control	AVE-0032	✓ sqlite ≥3.0.1
CVE-2025-67644	7.3 H	LangGraph	Mixed Control	AVE-0032	✓ sqlite ≥3.0.1
CVE-2026-27794	6.6 M	LangGraph	Deserialization	AVE-0041	✓ checkpoint ≥4.0
CVE-2026-28277	6.8 M	LangGraph	Deserialization	AVE-0041	✓ langgraph ≥1.0.10
CVE-2026-27022	Mod	LangGraph	Mixed Control	AVE-0032	✓
CVE-2025-68664	9.3 C	LangChain	Credential	AVE-0037	✓ core ≥0.3.81
CVE-2025-68665	High	LangChain JS	Credential	AVE-0037	✓
CVE-2025-65106	High	LangChain	Mixed Control	AVE-0019	✓

CVE	CVSS	Framework	Anti-Pattern	AVE Mapping	Patched
CVE-2026-34070	7.5 H	LangChain	Credential	AVE-0028	✓ core ≥1.2.22
CVE-2026-40087	5.3 M	LangChain	Mixed Control	AVE-0030	✓ core ≥0.3.84
CVE-2026-26013	Low	LangChain	Credential	AVE-0032	✓
CVE-2026-40190	Mod	LangSmith	Mixed Control	AVE-0019	✓
CVE-2026-2275	9.7 C	CrewAI	Sandbox	AVE-0038	✓
CVE-2026-2287	9.8 C	CrewAI	Sandbox	AVE-0038	✓
CVE-2026-2286	9.8 C	CrewAI	Credential	AVE-0032	✓
CVE-2026-2285	7.5 H	CrewAI	Credential	AVE-0028	✓
CVE-2025-5120	Crit	smolagents	Sandbox	AVE-0017	✓
CVE-2025-14931	Crit	smolagents	Deserialization	AVE-0041	✓
CVE-2025-11844	Mod	smolagents	Mixed Control	AVE-0030	✓
CVE-2026-2654	Mod	smolagents	Confused Deputy	AVE-0026	✓
CVE-2026-4963	Mod	smolagents	Confused Deputy	AVE-0026	✓
CVE-2026-40153	7.4 H	PraisonAI	Credential	AVE-0039	✓ ≥1.5.128
CVE-2026-40159	5.5 M	PraisonAI	Credential	AVE-0040	✓ ≥4.5.128

CVE	CVSS	Framework	Anti-Pattern	AVE Mapping	Patched
GHSA-ffp3	5.5 M	PraisonAI	Approval	AVE-0039	≥4.5.128
CVE-2026-40112	Mod	PraisonAI	Mixed Control	AVE-0030	

NAIL Institute — Neuravant AI Limited — nailinstitute.org